

Swagger Test-Anleitung: D&D VTT API (.NET 10 / Option B)

Diese Anleitung führt dich Schritt für Schritt durch das Testen deiner Endpunkte in der Swagger-UI (<https://localhost:<port>/swagger>).

Durch die Entkopplung von **Charakteren** (Kampagne) und **Tokens** (Spielfeld) entspricht der Testablauf dem echten Spielfluss eines Virtual Tabletops:

1. Compendium prüfen —> 2. GameSession erstellen —> 3. Karte hochladen
|
▼
4. Charakter anlegen —> 5. Spieler join Session —> 6. Token platzieren
|
▼
7. Token bewegen / HP <— 8. Zauber wirken <— 9. Grid anpassen

Vorbereitung: Backend starten

1. Stelle sicher, dass die Datenbank aktuell ist:

```
dotnet ef database update
```

2. Starte das Backend im Terminal:

```
dotnet run
```

3. Öffne im Browser die angezeigte Swagger-Adresse (<https://localhost:5089/swagger>).

Tipp für Swagger: Notiere dir die zurückgegebenen IDs (`id`, `joinCode`, `mapId`, etc.) kurz in einem Text-Editor, da spätere Endpunkte diese als Parameter benötigen.

Test-Schritte in logischer Reihenfolge

Schritt 1: Compendium prüfen (D&D 5e Daten)

Stelle sicher, dass deine SQLite-Datenbank ausgelesen werden kann:

1. Klappe GET `/api/Compendium/spells` auf.
 2. Klicke auf **Try it out**, trage bei `query` `fireball` ein und klicke auf **Execute**.
 3. **Erwartetes Ergebnis (200 OK):** Du erhältst das JSON für den Zauber *Fireball* (Index: `"fireball"`).
 4. (Optional) Wiederhole das mit GET `/api/Compendium/monsters?query=goblin` (Index: `"goblin"`).
-

Schritt 2: GameSession erstellen (Lobby)

Erstelle die Spielsitzung, die als Raum für alle Spieler und Karten dient.

1. Öffne POST `/api/GameSession`.
2. Klicke auf **Try it out** und füge folgende Nutzlast ein:

```
{
  "sessionName": "Kampagne: Berufsschule",
  "dungeonMasterId": "dm-Damian",
  "dungeonMasterName": "Dungeon Master Damian"
}
```

3. Klicke auf **Execute**.
 4. **Ergebnis (201 Created):**
 - Kopiere dir die `id` der Session (z. B. `3fa85f64-5717-4562-b3fc-2c963f66afa6`).
 - Kopiere dir den 6-stelligen `joinCode` (z. B. `K8X2W9`).
-

Schritt 3: Battlemap hochladen & aktiver Karte zuweisen

1. Öffne POST `/api/Map`.
 2. Klicke auf **Try it out**:
 - Title: Taverne zum Schlafenden Drachen
 - InitialCellSize: 70
 - image: Wähle ein beliebiges Bild von deiner Festplatte (.png oder .jpg).
 3. Klicke auf **Execute**.
 4. **Ergebnis (201 Created)**:
 - Kopiere dir die `id` der Map.
 - Das Backend hat automatisch ein passendes Grid in Grids erzeugt.
 5. **Karte für die Session aktivieren**:
 - Öffne PATCH `/api/GameSession/{id}/active-map`.
 - Trage bei `id` die **Session-ID** aus Schritt 2 ein.
 - Trage bei `newMapId` die **Map-ID** aus Schritt 3 ein.
 - Klicke auf **Execute (204 NoContent)**.
-

Schritt 4: Helden-Charakter erstellen

Der Charakter gehört zur Session/Kampagne und besitzt noch keine Spielfeld-Koordinaten.

1. Öffne POST `/api/Character`.
2. Klicke auf **Try it out** und nutze folgendes JSON:

```
{
  "gameSessionId": "<DEINE_SESSION_ID_HIER>",
  "name": "Damian der Krieger",
  "tokenImageUrl": "https://example.com/valeros.png",
  "ownerPlayerId": "player-stephan",
  "classIndex": "fighter",
  "raceIndex": "human",
  "maxHp": 28,
  "armorClass": 16,
  "speedInFeet": 30
}
```

3. Klicke auf **Execute**.

4. **Ergebnis (201 Created)**: Kopiere dir die `id` des Charakters.

Schritt 5: Spieler tritt der Session bei

Ein Spieler betritt die Lobby mit dem Join-Code und wählt seinen erstellten Helden aus.

1. Öffne `POST /api/GameSession/join`.

2. Klicke auf **Try it out**:

```
{
  "joinCode": "<DEIN_6_STELLIGER_JOIN_CODE>",
  "playerId": "player-Damian",
  "playerName": "Damian",
  "characterId": "<DEINE_CHARACTER_ID_AUS_SCHRITT_4>"
}
```

3. Klicke auf **Execute**.

4. **Ergebnis (200 OK)**: Die Session enthält den Spieler nun in der Teilnehmerliste (participants).

Schritt 6: Tokens auf der Battlemap platzieren

Nun wird der Charakter vom DM oder Spieler auf der aktiven Karte abgesetzt.

A. Spieler-Charakter platzieren

1. Öffne POST `/api/Token/map/{mapId}/place-character`.

2. Parameter:

- `mapId`: Deine Map-ID.
- `sessionId` (Query-Parameter): Deine Session-ID (für SignalR-Broadcast).

3. JSON Body:

```
{
  "characterId": "<DEINE_CHARACTER_ID_AUS_SCHRITT_4>",
  "gridX": 5,
  "gridY": 8
}
```

4. Klicke auf **Execute** (200 OK). Kopiere dir die `id` des erstellten Tokens.

B. Ein Monster aus dem Compendium spawnen

1. Öffne POST `/api/Token/map/{mapId}/spawn-monster`.

2. Parameter: `mapId` eintragen.

3. JSON Body:

```
{
  "monsterIndex": "goblin",
  "gridX": 10,
  "gridY": 8,
  "customName": "Goblin Späher"
}
```

4. Klicke auf **Execute** (200 OK). Das Backend übernimmt automatisch HP und AC des Goblins.
-

Schritt 7: Token bewegen und Schaden zufügen

1. Token bewegen:

- Öffne PATCH `/api/Token/{id}/move`.
- `id`: Die Token-ID aus Schritt 6.
- Body:

```
{
  "targetGridX": 6,
  "targetGridY": 8
}
```

- Klicke auf **Execute** (200 OK).

2. Schaden zufügen (HP anpassen):

- Öffne PATCH `/api/Token/{id}/hp`.
- Body:

```
{
  "newHp": 22
}
```

- Klicke auf **Execute** (200 OK).
-

Schritt 8: Zauber wirken (AoE-Schablone)

Ein Zauber wird auf die Karte gewirkt (z. B. Feuerball oder Nebelwolke).

1. Öffne POST `/api/Spell/map/{mapId}/cast`.
2. Parameter: `mapId` eintragen.

3. JSON Body:

```
{
  "spellIndex": "fireball",
  "casterTokenId": "<DEINE_TOKEN_ID_AUS_SCHRITT_6A>",
  "targetGridX": 10,
  "targetGridY": 8,
  "customRadiusFeet": 20
}
```

4. Klicke auf **Execute**.

5. **Ergebnis (200 OK):**

- Liefert das Berechnungs-Event mit Radius in Rasterzellen (`radiusInCells: 4`).
- Bei anhaltenden Konzentrationszaubern (z. B. `fog-cloud`) wird automatisch ein Schablonen-Eintrag in der Datenbank angelegt, den du über `GET /api/Spell/map/{mapId}/active` wieder abrufen kannst.

Schritt 9: Rastereinstellungen (Grid) anpassen

Passe die Rastergröße oder Linienfarbe für die Battlemap an:

1. Öffne `PUT /api/Grid/map/{mapId}`.

2. Parameter: `mapId` eintragen.

3. JSON Body:

```
{
  "cellSizeInPixels": 80,
  "lineColor": "#FFFFFF",
  "lineOpacity": 0.5,
  "offsetX": 5,
  "offsetY": 5
}
```

4. Klicke auf **Execute** (200 OK). Spalten- und Zeilenzahlen wurden automatisch neu berechnet.

Häufige Fehlerquellen & Schnelldiagnose

Statuscode	Typische Ursache	Lösung
400 Bad Request	Ungültiges GUID-Format oder falscher Index	Prüfe, ob die GUID vollständig kopiert wurde und ob Indizes (z. B. "fireball", "fighter") kleingeschrieben sind.
404 Not Found	ID existiert nicht in der Datenbank	Vergewissere dich, dass die referenzierte Map-, Session- oder Token-ID zuvor erfolgreich angelegt wurde.
415 Unsupported Media Type	POST /api/Map falsch angesprochen	Für den Map-Upload muss multipart/form-data genutzt werden (Swagger setzt das beim Datei-Upload automatisch).